

ch3 软件过程模型
软件过程是为建造高质量软件所需完成的任务的框架，即形成软件产品的一系列步骤，包括中间产品、资源、角色以及过程中采取的方法、工具等范畴
过程 是活动的集合，活动包括任务、工件（交付物）、质量保证点（要求）、里程碑
三组活动 ：基本软件开发活动（需求分析、软件设计、代码编写、软件测试、软件交付、软件进化）、软件工程过程框架活动（沟通、策划、建模、构建、部署）、保护性活动（项目跟踪和控制、风险管理、质量保证、技术评审、软件度量、软件配置管理、复用管理、工作产品的准备和生产）
经典软件过程模型
模型 要点 / 适用
瀑布 线性、文档驱动；需求 明确稳定 ，质量要求高
原型 快速原型探需求；需求 不明确
增量 需求较为清晰，功能并行、先做高风险，分批交付、核心先行；可分模块；有严格交付期限、资源有限
迭代 小瀑布的集合，短反馈、风险控制、管理要求高，需求持续变更、高风险，先做出落后优化
演化 迭代式开发+增量交付 ，需求不确定、创新/探索型，逐步添加功能
螺旋 瀑布+原型+迭代+ 风险分析 ；大型高风险
RUP 用例驱动 、以架构为中心、迭代增量

- **RUP 四阶段**：初始→细化→构造→移交。
- 只有瀑布模型和增量模型是应对清晰的需求的，别的全是不清晰，清晰的需求用户参与度就低，别的中高
- 增量交付：可以开发完核心功能后快速上线，迭代式开发：可以吸收经验应用到下一个迭代中

ch4 软件开发方法

结构化开发方法：更适合清晰稳定的需求，但是变化易在全局传播，一开始设计数据流，后续又要拆成模块，有鸿沟**面向对象开发方法**：适合大型、复杂、需长期维护的系统，面向对象分析和面向对象设计过度平滑

敏捷开发方法：**原则**：尽早且持续交付有价值的产品，欢迎变更，业务人员与开发人员在一起工作，面对面交谈，可用作指标，简洁；**价值观**：个体与互动>过程工具；可运行软件>详尽文档；客户合作>合同谈判；响应变化>遵循计划典型的敏捷开发方法：

- 极限编程 XP（特色：结对编程）：沟通，简单，反馈（测试），尊重，勇气
- Scrum 方法：若干迭代周期组成，每个周期称为冲刺，产品经理将产品分解为有优先级的产品任务列表->取出一定任务作为一次冲刺（此时冻结，需求不能变了）->开总会等->这个冲刺结束之后再循环下一个冲刺

为什么 OO 方法**填平**了结构化方法的两道**鸿沟**：

1. 封装把操作和数据绑定，填平“数据/行为鸿沟”
 2. OOA 与 OOD 用一致的表示法，无需大规模转换，填平了“分析/设计鸿沟”
- 有人认为软件工程浪费时间，**敏捷**的开发出代码才最重要？
1. 软件工程恰恰是为了解决“只写代码”的失败而诞生的。
 2. “能运行的代码”≠ 正确、可靠、可维护的软件。（要有需求分析、设计以及质量测试等）
 3. 省略前期工作的代价会在后期指数级放大。
 4. 文档与计划支撑协作、维护与知识传承。
 5. 把敏捷等同于“只写代码”，是对敏捷的根本误读。（敏捷是有计划、过程、纪律性工程实践、设计以及必要的文档的，只是反对官僚的、过度繁琐的流程和无效的文档）

关于**敏捷开发**意味着可以不需要文档、设计和计划：

1. 计划：敏捷计划的更频繁，比如上面 Scrum 的流程，XP 的计划游戏等。可见敏捷不是“不计划”，而是把“一次性详尽计划”换成了滚动式、分层、频繁重规划——计划的频率反而更高。
2. 设计：敏捷做的是“持续设计”，而非“不设计”。XP 包含简单设计，隐喻，CRC 卡片等设计技术，以及持续重构。反对的是大而全的前期设计，主张演进式设计。
3. 文档：敏捷保留“刚好够用+活文档”。比如 TDD 的测试，产品代办列表本身

ch2 软件工程概述

软件工程是一门研究如何用系统化、规范化、可量化等工程原则和方法进行软件开发和维护的学科。其目标是创建“足够好”、用户满意的软件。不仅关注软件开发的技术过程，也关注软件项目管理以及支持软件开发的工具、方法和理论

七条原则：1.使用分阶段的生命周期计划进行管理 2.坚持进行阶段评审 3.实行严格的产品控制 4.采用现代编程实践 5.对结果保持明确的问责制 6.开发团队应该少而精 7.软件过程持续改进

软件危机：**表现**：1.所交付软件不符合用户的需求 2.软件质量不可靠，得不到用户的信赖 3.项目超预算或延误。**根源**：1.软件复杂度 2.软件开发技术问题 3.组织和管理问题 4.市场需求问题（需求变化快，推着对开发要求也更快）5.缺乏标准和规范

计算机辅助软件工程的工具：辅助软件工全生命周期的开发工具和方法集合 1.系统分析与设计工具（UML） 2.程序设计工具（IDEA） 3.软件测试与质量保证工具（Java 压测工具 JMeter） 4.软件项目管理工具（GitHub） 5.软件运维工具（Docker）

软件工程三要素：过程（管理视角）、方法学（技术视角）、工具（辅助视角）

ch1 软件

软件=程序+数据+文档

特点：复杂性、一致性、易变性和不可见性

分类：系统软件（编译器和硬件驱动也属于这类）和应用软件（为了某种用途被开发）另一种分类方式：关键软件：关键基础软件（os、db、中间件、办公）、大型工业软件、行业应用软件（金融）、新型平台软件、嵌入式软件

软件生存周期(SLC)：软件开发生存周期(SDLC)+交付后的维护阶段。软件产品生存周期：包含多个 SDLC，持续改进和增强。阶段：

1. 软件定义时期：问题定义、可行性研究、需求分析
2. 软件开发时期：总体设计、详细设计、编码和单元测试、综合测试
3. 软件运行维护时期（工作量占比最大）

软件质量模型：（质量衡量标注）

1. McCall 模型：3 个维度 11 个质量因素，目的是避免开发用户不想要的模型，产品运行：正确性、可靠性、效率、安全性、可用性；产品修改：可维护性、灵活性、可测试性；产品转移：可移植性、可重用性、互操作性
2. Garvin 模型：8 个质量维度，目的是“主动提供用户真正想要的产品”，性能、特性、可靠性、符合性、耐用性、服务性、美学性、感知质量
3. ISO/IEC 25010: 8 大特性、31 个子特性，现行国际标准，最系统最完整，功能适合性、性能效率、兼容性、易用性、可靠性、信息安全性、维护性、可移植性

软件质量保证技术(SQA)：

静态（无需运行程序，尽早发现问题、成本低）：

- ① 质量标准制定：编程/设计/测试/文档等规范，贯穿全程
- ② 阶段评审：人工审查需求、设计、代码（需求评审、设计评审、代码走查）
- ③ 软件静态分析：工具自动扫描代码结构/语法/语义（如 SonarQube、FindBugs）

动态（需运行程序，验证真实行为、成本较高）：

- ④ 软件测试：实际运行验证（单元/集成/系统/验收测试）
- ⑤ 代码重用：复用已验证的代码，降低引入缺陷的风险

ch8 软件测试

1. 静态测试：代码走查（问问思路）、技术评审、代码审查
2. 动态测试：1.是否关心内部实现（白盒/黑盒/灰盒测试） 2.按开发过程阶段（单元/集成/系统/验收/回归测试） 3.按实施组织：开发方测试（α测试）、用户测试（β测试）、第三方测试

软件测试的七条准则：1.测试可以证明缺陷存在，但不能证明不存在 2.穷尽测试是不可能的 3.测试活动应该尽早开始 4.缺陷集群性 5.杀虫剂悖论（一组测试用例一直用会无效） 6.测试依赖于测试内容 7.“没有失效就是有用系统”是一种谬论

白盒测试：（要求了解代码内部逻辑）

语句覆盖（每条语句执行一次）->判定覆盖（每个判定的每种结果至少一次）->条件覆盖（每个条件（原子）的真假至少各一次）->判定-条件覆盖（同时满足判定覆盖和条件覆盖）->条件组合覆盖（每个判定中所有组合出现一次，比如 AorB，AB 就要分别出现 TT、TF、FT、FF）->路径覆盖（覆盖程序流程图中的所有路径）

基本路径覆盖：至少引入一个新语句或者新判断的程序通道

1. 程序控制流图绘制：判定点、过程块、结合点、结束点，注意判定点不应包含符合条件，如果包含需要拆分为两个点
2. 计算圈复杂度：E(dges) - N(nodes) + 2，结果为基本路径数
3. 给复合条件标两个节点，括号内多句话挂成一个节点，单个条件也分配一个节点，单句话分配一个节点，**终止点也有一个节点，比如 while，从开始直接指向这个终止点就是一条基本路径**
4. 可以适当标注一些 T 和 F 和入口出口等辅助自己理解
5. 提到满足基本路径覆盖的用例，就标注每条路径对应的测试用例，如果有某条路径无法经过，也要对应说明

黑盒测试：（不看内部，仅关心输入输出）

1. 等价类划分法：把输入域划分为若干集合，每个集合中只选一个，分为**有效等价类**和**无效等价类**。测试用例设计原则：**一个用例尽量覆盖所有有效类**；每个无效类单独一个用例（**每次只让一项非法**、其余保持合法）。**这个等价类可以划分的细一点**
2. 边界值分析法：找正好等于，刚刚大于或刚刚小于边界的值、空值来测试
3. 错误推测法：根据经验来
4. 判定表法：分条件桩（一些条件）和动作项（结果是什么）图表

按照阶段分类：

级别	阶段	对象	人员	依据	方法
单元测试	编码后	最小可测单元	开发者	代码/详细设计文档	白盒
集成测试	单元后	模块接口/协作	开发者	概要设计文档	黑+白
确认测试	集成后(冒烟过)	系统	测试员+用户	需求规格说明书	黑盒
系统测试	确认后	整个系统(软硬件)	测试员+用户	系统说明书/需求文档	黑盒
验收测试	发布前	整个系统	用户/需求方	需求/验收标准	黑盒

流图、用例图、顺序图、类图等

导出方式：ctrl+shift+p，export current diagram

流图：

@startuml
' 为了让图形更紧凑，可以设置方向从上到下
top to bottom direction

```
' 定义圆形节点
usecase "1" as n1
usecase "2" as n2
usecase "3" as n3
```

```
' 绘制边和方向
n1 --> n2
n2 --> n3 : T
```

@enduml

注意：

1. 复合条件需要拆开
2. 从起始点直接到终止点的路径不要忘记

用例图：

```
@startuml
树洞APP用例图
left to right direction
actor "匿名用户" as User
actor "审核员" as Auditor
actor "用户" as UUser
```

```
rectangle 匿名树洞APP {
    usecase "匿名登录" as UC00
    usecase "发布内容" as UC10
    usecase "AI情绪疏导" as UC11
    usecase "内容审核" as UC20
}
```

```
' 泛化关系
Auditor -->|UUser
User -->|UUser
```

```
UC10 ..> UC00 : <<include>>
UC11 ..> UC10 : <<extend>>
```

```
' 如果用--就不强调角色发起用例的关系
User --> UC10
Auditor --> UC20
```

@enduml

注意：

1. 不是提到了的角色都要化成参与者，比如保洁用钥匙，不经过系统，就不需要体现在图上
2. 一些外部系统，比如：学校统一身份认证系统、短信系统也要画
3. 所有角色都要做的，比如身份认证，可以抽象出一个父角色，让他做，而不必所有人都去指一遍，泛化（子指向父）

顺序图：

- boundary/control/entity：边界/控制/实体类
- -->同步消息，--->返回消息
- activate/deactivate：激活/销毁生命周期
- alt/opt/loop/par：条件/可选/循环/并行
- ref over：嵌套子流程（复合顺序图）
- autonumber：自动编号

```
@startuml
发布内容顺序图
autonumber
actor "用户" as User
boundary "发布UI" as UI <<boundary>>
control "内容控制器" as Manager <<controller>>
entity "内容实体" as Post <<entity>>
```

```
User -> UI : 提交内容
activate UI
UI -> Manager : 发起发布请求
activate Manager
```

```
' 嵌套子流程
ref over Manager, Post : 权限校验子流程
```

```
' 条件分支
alt 权限通过
    Manager -> Post : 保存内容
    Post --> Manager : 成功
    Manager --> UI : 发布成功
else 权限失败
    Manager --> UI : 发布失败
end
```

```
deactivate Manager
deactivate UI
@enduml
par
    ATM -> 银行系统 : 记录交易日志
else
    ATM -> 用户 : 返回主菜单
end
opt 用户选择打印凭条
    ATM -> 用户 : 打印凭条
end
loop 密码错误，最多3次
    ATM -> Bank : 验证密码(卡号，pwd)
    activate Bank
    Bank --> ATM : 返回验证结果
    deactivate Bank
```

end

注意：

1. 预约界面，预约控制器，用户账号，会议室，预约记录，短信系统
2. 从上到下是完成某个操作随着时间的各种行为

类图：

```
@startuml
class BankAccount {
    - id : String
    - balance : double
    - {static} interestRate : double
    ..
    + deposit(amount : double) : void
    + getBalance() : double
    + {abstract} calculate() : double
}
@enduml
#protected, ~package
特殊类型：抽象类、接口、枚举
@startuml
abstract class Shape {
    + {abstract} area() : double
}

interface Drawable {
    + draw() : void
}

enum OrderStatus {
    PENDING
    PAID
    SHIPPED
    COMPLETED
}
@enduml
```

六种关系

泛化/继承：子类指向父类 B --|> A

实现：实现类指向接口 B ..|> A

接下来关联、聚合、组合、依赖

关系强弱程度（耦合程度）：依赖..>关联--/--><聚合 o--<组合*--

组合：整体消亡，部分同步消亡

聚合：部分可以独立存在

如果偶尔传进来用一下是依赖，作为字段一直持有是关联

多重性**必须标注**：写在关系线两端，表示一个对象对应另一端的数量

Customer "1" --> "0..*" Order : places >

Teacher "1" -- "1..*" Student : teaches

注意：

1. 可以直接用汉字写一些函数名和变量名
2. 可以利用上面顺序图的拆解方式直接画类图

ch9 软件部署与维护

软件维护：软件交付使用后，对软件系统及其配套设施进行修改的过程,目的是修复缺陷、提高性能或其他属性、增强功能以及适应变化的环境。

软件的可维护性：软件交付使用后，为满足新要求而进行有效修改、扩展和优化的能力（含修复缺陷、改进设计、增加功能、适应新环境）。

影响软件可维护性的因素：1.可理解性：结构、组件、代码易于理解和解释 2.可测试性：测试易进行，能检出错误 3.可修改性：易修改与扩展，满足新需求 4.可移植性：能在不同环境部署运行 5.可重用性：具有可重用组件

软件维护的形式和类别：

类型	驱动	目的	关键特点
纠正性	已暴露的缺陷	修复 bug、保证正常运行	被动性 、紧急性
改善性	内部求更好 / 新需求	提性能、增功能、优体验	功能增强、性能优化、UX 提升
适应性	外部环境变化	与新环境兼容	环境适应、兼容性测试、更新频繁
预防性	为未来风险	提可维护性/可靠性、延长寿命	前瞻性 、持续性、系统性

软件维护的任务：旨在确保软件全生命周期保持高质量、高性能、高可用。主要任务：问题诊断、缺陷修复、功能更新、性能优化、兼容性适应、代码重构、文档更新、安全性维护、用户支持、软件测试。

软件维护的过程：问题收集和分析、提出更改请求、设计修改方案、评审修改方案、实施修改方案、软件测试、软件发布和部署、监控和用户反馈、调整和优化

软件维护的原则：风险可控、主体维持、系统总体结构优化、复杂性可控

软件维护的实施策略：

建立规范的流程；分类实施（按四类维护分别处理）；强化技术支持与工具链建设；组织保障与团队协作机制；推动从“被动修复”到“主动预防”

软件维护技术：

1. DOM 面向缺陷：目标：修缺陷、降故障率。五阶段：识别→分类→定位→修复→验证。适用**高质量/高可靠**；不适用功能简单、短生命周期。
2. FOM 面向功能：目标：满足需求、改功能（新功能/改进/兼容/测试）。四阶段：分析→设计→修改→测试。适用**功能复杂/长生命周期**。
3. 软件再工程：重构改造旧系统，提“四性”（可维护/重用/扩展/移植），**功能性能不变**。
 - **库存分析**：建系统清单(规模/修改数/文档质量/业务价值…)，定期修订。
 - **四象限**：**低质量+高价值→再工程**；高质高值→正常维护；低价值→一般维护或弃。
 - **三类重构**：文档 / 代码 / 数据库。
 - **坏味道(高频)**：重复代码、过长函数、过大类、过长参数列。

概念	方向	做什么
逆向工程	低→高	由代码反推设计/需求与文档
重构/重组	同层	改善结构，不改抽象层次
正向工程	高→低	重建系统并集成新需求/技术

再工程 = 逆向(理解旧系统) + 重构/改造 + 正向(重建并增强)

ch6 软件设计